

“Año de la Esperanza y el Fortalecimiento de la Democracia”

UNIVERSIDAD PERUANA LOS ANDES



FACULTAD INGENIERIA
ESPECIALIDAD: SISTEMAS Y COMPUTACION

Desarrollo Ejercicios _Semana 11

CURSO:

- Base de Datos II

DOCENTE:

- Fernández Bejarano Raúl Enrique

ESTUDIANTE:

- Rosales Crispín Aristóteles Onassis

Ciclo:

- 5°SEMESTRE

HUANCAYO- JUNIN
2026

TEMA 1: Autenticación SQL y Windows: diferencias y prácticas seguras

Proyecto 1: Implementación de acceso mixto para personal académico y administrativo

Enunciado:

La Universidad requiere que los trabajadores internos accedan mediante autenticación Windows, mientras que los consultores externos utilicen autenticación SQL Server. Diseñe e implemente una estrategia de autenticación para la base de datos (GradosTitulos) que permita administrar ambos tipos de acceso.

```
USE [master];
GO

-- 1. Habilitar el modo de autenticación mixta (debe hacerse a nivel del
servidor)
-- Este paso se realiza mediante SQL Server Management Studio o con T-SQL:
EXEC xp_instance_regwrite N'HKEY_LOCAL_MACHINE',
    N'Software\Microsoft\MSSQLServer\MSSQLServer',
    N'LoginMode', REG_DWORD, 2; -- 2 = Mixed Mode
GO

-- 2. Crear logins de Windows para personal interno (usuarios de dominio)
-- Nota: Los nombres de los logins deben coincidir con los del Active Directory
-- Ejemplo para el personal de la Oficina de Grados y Títulos:
CREATE LOGIN [DOMINIO\coordinador_gt] FROM WINDOWS;
CREATE LOGIN [DOMINIO\secretario_gt] FROM WINDOWS;
CREATE LOGIN [DOMINIO\decano_facultad] FROM WINDOWS;
-- Se pueden agregar más usuarios según sea necesario.

-- 3. Crear logins de SQL Server para consultores externos
-- (Asignar contraseñas seguras y cumplir políticas de complejidad)
CREATE LOGIN consultor_ext1
    WITH PASSWORD = 'Up1a#2026$Seguro!',
    CHECK_POLICY = ON,
    CHECK_EXPIRATION = ON,
    DEFAULT_DATABASE = GradosTitulos;

CREATE LOGIN consultor_ext2
    WITH PASSWORD = 'ExtConsultor*2026',
    CHECK_POLICY = ON,
    CHECK_EXPIRATION = ON,
    DEFAULT_DATABASE = GradosTitulos;

-- 4. Usar la base de datos GradosTitulos
USE GradosTitulos;
GO

-- 5. Crear usuarios de base de datos a partir de los logins
-- Para usuarios Windows
CREATE USER [coordinador_gt] FOR LOGIN [DOMINIO\coordinador_gt];
CREATE USER [secretario_gt] FOR LOGIN [DOMINIO\secretario_gt];
CREATE USER [decano_facultad] FOR LOGIN [DOMINIO\decano_facultad];

-- Para consultores externos
```

```

CREATE USER consultor1 FOR LOGIN consultor_ext1;
CREATE USER consultor2 FOR LOGIN consultor_ext2;

-- 6. Definir roles preexistentes o crear nuevos según perfiles
-- (Asumimos que existen roles como 'CoordinacionGT', 'AdminTramite', etc.)
-- Si no existen, se crean:
CREATE ROLE CoordinacionGT;
CREATE ROLE ConsultorExterno;

-- 7. Asignar permisos a los roles
-- Ejemplo: CoordinacionGT tiene acceso de lectura/escritura a las tablas de trámite
GRANT SELECT, INSERT, UPDATE, DELETE ON Tramite.Tramite TO CoordinacionGT;
GRANT SELECT, INSERT, UPDATE, DELETE ON Tramite.CoordinacionGT TO
CoordinacionGT;
GRANT SELECT ON Academico.Matricula TO CoordinacionGT;
GRANT SELECT ON Academico.Persona TO CoordinacionGT;

-- Ejemplo: ConsultorExterno solo tiene acceso de lectura a algunas vistas
GRANT SELECT ON Tramite.V_EstadoTrabajo TO ConsultorExterno;
GRANT SELECT ON Evaluacion.V_NotaFinalSustentacion TO ConsultorExterno;
-- Se restringe el acceso a tablas sensibles

-- 8. Asignar usuarios a los roles
ALTER ROLE CoordinacionGT ADD MEMBER [coordinador_gt];
ALTER ROLE CoordinacionGT ADD MEMBER [secretario_gt];
ALTER ROLE CoordinacionGT ADD MEMBER [decano_facultad];

ALTER ROLE ConsultorExterno ADD MEMBER consultor1;
ALTER ROLE ConsultorExterno ADD MEMBER consultor2;

-- 9. Revocar permisos por defecto (principio de mínimo privilegio)
-- Denegar cualquier permiso adicional a PUBLIC
REVOKE SELECT ON Academico.Persona TO PUBLIC;
REVOKE SELECT ON Tramite.Tramite TO PUBLIC;
-- (Asegurarse de que los usuarios solo tengan lo que se les otorgó explícitamente)

-- 10. Opcional: Configurar auditoría para logins de SQL (se verá en proyecto 3)

```

Justificación Técnica

La autenticación Windows (integrada) es la opción más segura para los empleados internos porque delega la verificación de identidad al Active Directory del dominio. Esto elimina la transmisión de contraseñas por la red, aprovecha las políticas corporativas de bloqueo y caducidad, y usa el protocolo Kerberos, más robusto que el hash de contraseña de SQL Server. Además, la autenticación Windows facilita la administración centralizada y la trazabilidad de accesos (los eventos de seguridad se registran en los controladores de dominio). La autenticación SQL Server es necesaria para consultores externos que no pertenecen al dominio corporativo. El modo mixto (Mixed Mode) permite coexistir ambos esquemas, pero debe habilitarse explícitamente en SQL Server On-Premise. En este proyecto, se han creado logins SQL con CHECK_POLICY = ON y CHECK_EXPIRATION = ON para heredar las políticas de complejidad y caducidad de contraseñas definidas a nivel del sistema operativo, reduciendo el riesgo de contraseñas débiles o sin rotación. Se ha aplicado el principio de mínimo privilegio: cada usuario solo obtiene los permisos necesarios para su rol. Los empleados de la coordinación tienen permisos de manipulación sobre las tablas de trámite, mientras que los consultores externos solo pueden consultar vistas predefinidas (no tablas base) y no tienen acceso a datos sensibles (como información personal de los estudiantes).

Buenas Prácticas

- Preferir autenticación Windows siempre que sea posible; reservar SQL Auth solo para casos donde Windows Auth no sea viable técnicamente (por ejemplo, equipos sin dominio o usuarios externos).
- Activar CHECK_POLICY = ON y CHECK_EXPIRATION = ON en todos los logins SQL para forzar contraseñas robustas y su renovación periódica, alineadas con las políticas de seguridad de la organización.
- No compartir un login entre múltiples personas; cada usuario real debe tener su propio login para garantizar trazabilidad en auditorías y facilitar la revocación de accesos.
- Definir roles personalizados (como CoordinacionGT y ConsultorExterno) para agrupar permisos y simplificar la administración, en lugar de asignar permisos directamente a los usuarios
- Revocar permisos implícitos (PUBLIC) para evitar que usuarios no autorizados accedan a objetos de la base de datos por defecto.
- Documentar y auditar todos los logins creados, así como los cambios en los roles y permisos, mediante herramientas de auditoría nativas o de terceros.

Proyecto 2: Gestión de accesos para miembros del Jurado de Tesis

Enunciado: Los miembros del jurado necesitan acceder únicamente a información de tesis y sustentaciones. Configure accesos diferenciados utilizando autenticación Windows y SQL.

```
USE [master];
GO

-- 1. Crear logins para miembros del jurado (algunos internos vía Windows,
-- otros externos vía SQL)
-- Miembros internos (docentes de la facultad) con autenticación Windows
CREATE LOGIN [DOMINIO\docente_jurado1] FROM WINDOWS;
CREATE LOGIN [DOMINIO\docente_jurado2] FROM WINDOWS;

-- Miembros externos (por ejemplo, profesionales invitados) con autenticación
SQL
CREATE LOGIN jurado_externo1
    WITH PASSWORD = 'Jurado#2026!Externo',
    CHECK_POLICY = ON,
    CHECK_EXPIRATION = ON,
    DEFAULT_DATABASE = GradosTitulos;

CREATE LOGIN jurado_externo2
    WITH PASSWORD = 'Externo@Jurado2026',
    CHECK_POLICY = ON,
    CHECK_EXPIRATION = ON,
    DEFAULT_DATABASE = GradosTitulos;

-- 2. Usar la base de datos GradosTitulos
USE GradosTitulos;
GO

-- 3. Crear usuarios de base de datos
CREATE USER [docente_jurado1] FOR LOGIN [DOMINIO\docente_jurado1];
CREATE USER [docente_jurado2] FOR LOGIN [DOMINIO\docente_jurado2];
CREATE USER jurado_ext1 FOR LOGIN jurado_externo1;
CREATE USER jurado_ext2 FOR LOGIN jurado_externo2;

-- 4. Crear un rol específico para el jurado (lectura de tesis y sustentaciones)
CREATE ROLE RolJurado;

-- 5. Conceder permisos de solo lectura a las tablas y vistas necesarias
-- Tablas de trabajo de investigación y sustentaciones
GRANT SELECT ON Tramite.TrabajoInvestigacion TO RolJurado;
GRANT SELECT ON Tramite.DesignacionJurado TO RolJurado;
GRANT SELECT ON Tramite.DesignacionAsesor TO RolJurado;
GRANT SELECT ON Evaluacion.Sustentacion TO RolJurado;
GRANT SELECT ON Evaluacion.EvaluacionTrabajo TO RolJurado;
GRANT SELECT ON Evaluacion.ControlSimilitud TO RolJurado;
GRANT SELECT ON Evaluacion.DictamenEtica TO RolJurado;

-- Vistas auxiliares
GRANT SELECT ON Tramite.V_EstadoTrabajo TO RolJurado;
GRANT SELECT ON Evaluacion.V_NotaFinalSustentacion TO RolJurado;

-- Se restringe el acceso a tablas con información personal (Persona, Matricula,
etc.)
-- mediante la denegación explícita (principio de necesidad de conocer)
DENY SELECT ON Academico.Persona TO RolJurado;
```

```

DENY SELECT ON Academico.Matricula TO RolJurado;
DENY SELECT ON Tramite.Tramite TO RolJurado;  -- Solo verán a través de las
vistas

-- 6. Agregar los usuarios al rol
ALTER ROLE RolJurado ADD MEMBER [docente_jurado1];
ALTER ROLE RolJurado ADD MEMBER [docente_jurado2];
ALTER ROLE RolJurado ADD MEMBER jurado_ext1;
ALTER ROLE RolJurado ADD MEMBER jurado_ext2;

-- 7. Opcional: Para asegurar que solo vean sus propias designaciones (filtrado
por fila)
-- Se podría usar seguridad a nivel de fila (RLS), pero se omite por
simplicidad.

-- En su lugar, se confía en que la aplicación filtre por IdDocente.

```

Justificación técnica

Los miembros del jurado (docentes y profesionales externos) necesitan acceder a la información de las tesis y sustentaciones para cumplir su función evaluadora, pero no requieren ni deben ver datos administrativos sensibles (por ejemplo, notas de otros estudiantes, datos personales de los alumnos, estados de pago, etc.).

Por ello, se ha diseñado un rol RolJurado con permisos de solo lectura sobre las tablas y vistas directamente vinculadas al trabajo de investigación y su evaluación. Se aplica autenticación Windows para los jurados internos (docentes de la universidad) y SQL Server para los externos (invitados de otras instituciones), replicando el mismo enfoque mixto del proyecto 1.

Todos los logins SQL tienen políticas de contraseña activas. Se ha denegado expresamente el acceso a tablas que contienen información personal (como Persona y Matricula) y a la tabla principal Tramite, forzando a los jurados a consultar únicamente las vistas y tablas específicas que contienen la información necesaria para su labor. Esto reduce la superficie de ataque y cumple con la normativa de protección de datos.

Buenas prácticas

- Aplicar el principio de mínimo privilegio: los jurados solo tienen permisos SELECT sobre objetos estrictamente necesarios; no tienen permisos de INSERT, UPDATE o DELETE.

- Separar los roles por responsabilidad (jurado, coordinador, administrador) para facilitar la asignación y revocación de permisos sin afectar a otros perfiles.

- Utilizar autenticación Windows para usuarios internos y SQL para los externos, pero siempre con políticas de contraseña robustas en estos últimos.

-Denegar explícitamente el acceso a datos personales y administrativos, incluso si el rol no tiene permisos, para evitar posibles sobrepermisos heredados de roles superiores.

Documentar la asignación de permisos y revisar periódicamente que ningún miembro del jurado tenga accesos no autorizados.

Considerar el uso de vistas (como V_EstadoTrabajo) para exponer solo los campos necesarios y simplificar la consulta, evitando que los jurados tengan que navegar por uniones complejas.

Proyecto 3: Gestión de accesos para miembros del Jurado de Tesis

Enunciado:

La Oficina de Tecnologías de Información requiere aplicar políticas estrictas para las cuentas SQL Server que administran los trámites de grados y títulos.

```
USE [master];
GO

-- 1. Crear un login administrativo dedicado para la gestión de grados y
títulos
-- Este login será usado por aplicaciones o por personal de la oficina de TI.
CREATE LOGIN admin_grados
    WITH PASSWORD = 'Adm1n#Grados*2026!',
    CHECK_POLICY = ON,
    CHECK_EXPIRATION = ON,
    DEFAULT_DATABASE = GradosTitulos,
    DEFAULT_LANGUAGE = Spanish;

-- 2. Aplicar políticas adicionales a este login (y a otros logins SQL críticos)
-- a) Límite de intentos de conexión fallidos (mediante políticas de Windows o
trigger)
-- No se puede configurar directamente en SQL Server, pero se puede usar un
trigger de logon.
-- b) Restringir las conexiones solo desde direcciones IP específicas (usando
trigger de logon)
-- c) Habilitar la auditoría de inicios de sesión exitosos y fallidos

-- 3. Crear un trigger de logon para restringir conexiones solo desde la red
interna
-- (Ejemplo: solo se permite la IP 192.168.1.0/24)
CREATE OR ALTER TRIGGER trg_LogonRestriction
ON ALL SERVER
FOR LOGON
AS
BEGIN
    DECLARE @client_ip VARCHAR(15);
```

```

SET @client_ip = (SELECT client_net_address FROM sys.dm_exec_connections
WHERE session_id = @@SPID);

-- Permitir solo conexiones desde la subred interna
IF (ISNULL(@client_ip, '') NOT LIKE '192.168.1.%' AND @client_ip <> ':::1')
BEGIN
    ROLLBACK;
    -- Se puede registrar el intento en una tabla de auditoría (si existe)
END
END;
GO

-- 4. Habilitar la auditoría de logins (auditoría a nivel de servidor)
-- Crear una especificación de auditoría para registrar todos los eventos de
autenticación
CREATE SERVER AUDIT Audit_GradosTitulos
TO FILE (FILEPATH = 'C:\AuditLogs\')
WITH (ON_FAILURE = CONTINUE);

ALTER SERVER AUDIT Audit_GradosTitulos WITH (STATE = ON);

CREATE SERVER AUDIT SPECIFICATION Spec_Logins
FOR SERVER AUDIT Audit_GradosTitulos
ADD (SUCCESSFUL_LOGIN_GROUP),
ADD (FAILED_LOGIN_GROUP);
ALTER SERVER AUDIT SPECIFICATION Spec_Logins WITH (STATE = ON);

-- 5. Usar la base de datos GradosTitulos para crear el usuario administrativo
USE GradosTitulos;
GO

CREATE USER admin_grados FOR LOGIN admin_grados;

-- 6. Asignar los permisos necesarios para la administración completa
-- (Por ejemplo, ser miembro del rol db_owner, pero es preferible crear roles
personalizados)
EXEC sp_addrolemember 'db_owner', 'admin_grados';
-- O bien, otorgar permisos específicos a tablas y procedimientos:
/*
GRANT SELECT, INSERT, UPDATE, DELETE ON Tramite.Tramite TO admin_grados;
GRANT EXECUTE ON Tramite.SP_RegistrarTramite TO admin_grados;
GRANT EXECUTE ON Evaluacion.SP_RegistrarSustentacion TO admin_grados;
... etc.
*/

-- 7. Establecer límites de recursos para este login (si es necesario)
-- (Usar Resource Governor, aunque se omite por simplicidad)
-- ALTER LOGIN admin_grados WITH DEFAULT_DATABASE = GradosTitulos;

-- 8. Configurar la expiración de contraseña y bloqueo por intentos fallidos
-- (Esto ya está incluido en CHECK_POLICY = ON, que usa la política de Windows)
-- Para forzar el cambio de contraseña en el próximo inicio de sesión:
ALTER LOGIN admin_grados WITH PASSWORD = 'Adm1n#Grados*2026!' MUST_CHANGE;
-- Esto obliga al administrador a cambiar la contraseña al conectarse.

-- 9. Revocar permisos innecesarios a PUBLIC y a otros roles
REVOKE VIEW ANY DATABASE TO PUBLIC;
REVOKE VIEW SERVER STATE TO PUBLIC;
-- etc.

-- 10. Crear un procedimiento para monitorear accesos sospechosos
-- (Por ejemplo, verificar logins fallidos)

```



```

CREATE OR ALTER PROCEDURE Seguridad.SP_ReporteIntentosFallidos
AS
BEGIN
    SELECT
        event_time,
        server_principal_name,
        client_ip,
        [application_name],
        [data]
    FROM sys.fn_get_audit_file('C:\AuditLogs\*.sqlaudit', DEFAULT, DEFAULT)
    WHERE action_id = 'LGIF' -- Failed login
    ORDER BY event_time DESC;
END;
GO

```

Justificación técnica

Este proyecto está orientado a cuentas con alto privilegio (administradores de la aplicación o del sistema) que gestionan los trámites de grados y títulos. La Oficina de TI exige políticas estrictas para estas cuentas, ya que un compromiso de las mismas pondría en riesgo la integridad y confidencialidad de toda la base de datos.

Se ha creado un login específico admin_grados con las siguientes características:

- Políticas de contraseña (CHECK_POLICY = ON, CHECK_EXPIRATION = ON) para garantizar contraseñas complejas y caducidad periódica.
- Obligación de cambio (MUST_CHANGE) en la primera conexión, asegurando que el administrador asuma el control de su credencial.
- Restricción de origen mediante un trigger de logon que solo permite conexiones desde la subred corporativa, evitando accesos desde redes externas (incluso si la contraseña es comprometida).
- Auditoría exhaustiva de todos los intentos de inicio de sesión (tanto exitosos como fallidos) mediante una auditoría de servidor, almacenada en un directorio seguro para su posterior revisión.
- Permisos mínimos pero suficientes: aunque en este ejemplo se le asigna db_owner, en un entorno real se recomienda crear roles personalizados y otorgar solo los permisos estrictamente necesarios para las operaciones administrativas.

Además, se incluye un procedimiento para consultar el registro de auditoría y detectar intentos fallidos, lo que facilita la respuesta ante posibles ataques de fuerza bruta.

Buenas prácticas

- Usar cuentas dedicadas y no compartidas para tareas administrativas; cada administrador debe tener su propio login y no usar el mismo sa ni cuentas genéricas.
- Activar siempre CHECK_POLICY y CHECK_EXPIRATION en logins SQL, y forzar el cambio de contraseña periódicamente (especialmente después de incidentes).
- Restringir las conexiones a direcciones IP confiables (red interna, subredes autorizadas) para reducir la superficie de ataque.
- Auditar todos los inicios de sesión y almacenar los registros en un lugar seguro y de solo escritura, con rotación de archivos para evitar que se llenen.

- Aplicar el principio de mínimo privilegio incluso para administradores: no otorgar sysadmin a menos que sea absolutamente necesario; usar roles de base de datos (db_owner, db_ddladmin, etc.) de forma granular.
- Revisar periódicamente los accesos mediante scripts que comparen la membresía de roles con la plantilla de permisos aprobada.
- Documentar los cambios en las políticas de seguridad y mantener un registro de las cuentas administrativas, sus propósitos y los responsables.
- Implementar alertas para notificar a los administradores de TI ante múltiples intentos fallidos de inicio de sesión en un corto período.

TEMA 2: Cuentas de servicio y configuración del servidor

Proyecto 1: Configuración segura de servicios SQL Server
Enunciado: La base de datos GradosTitulos será utilizada en producción. Configure cuentas de servicio específicas para SQL Server y SQL Server Agent.

```
-- 1. Verificar las cuentas de servicio actuales (desde T-SQL)
-- Para SQL Server y SQL Server Agent, consultamos el registro o usamos DMV:
SELECT
    servicename,
    service_account,
    startup_type_desc,
    status_desc,
    process_id
FROM sys.dm_server_services;
GO

-- 2. Cambiar las cuentas de servicio (se debe hacer con SQL Server
Configuration Manager o PowerShell)
-- Recomendación: Usar una cuenta de dominio con privilegios mínimos, por
ejemplo: DOMINIO\svc_sqlserver y DOMINIO\svc_sqlagent.
-- Pasos:
-- a) Crear las cuentas de dominio en Active Directory (o locales si no hay
dominio).
-- b) Asignarles los permisos necesarios:
--   - Iniciar sesión como servicio (Logon as a service).
--   - Permisos sobre carpetas de datos, logs, backups.
--   - En SQL Server Configuration Manager, asignar la cuenta a cada servicio.
--   - Para SQL Server Agent, otorgar permisos para ejecutar trabajos.

-- 3. Verificar que los servicios estén configurados con las nuevas cuentas:
-- (Ejecutar nuevamente la consulta anterior)
```

Justificación técnica

En producción, es fundamental que los servicios de SQL Server y SQL Server Agent se ejecuten con cuentas de servicio dedicadas, con privilegios mínimos (principio de mínimo privilegio). Esto limita el daño potencial si la cuenta se ve comprometida, y facilita la auditoría y el control de acceso a recursos del sistema (archivos, red, etc.). Además, el uso de cuentas de dominio permite centralizar la gestión de contraseñas y políticas de grupo. Separar las cuentas de ambos servicios reduce el riesgo de que un problema en el Agente afecte al motor principal.

Buenas prácticas

- Usar cuentas de dominio con privilegios mínimos (no administradores locales) y con contraseñas robustas, rotadas periódicamente.
- Asignar permisos específicos a las carpetas de datos, logs, y backup (lectura/escritura solo para esas cuentas).
- Configurar el inicio automático de ambos servicios.
- Monitorear los servicios con alertas (ver proyecto 2).
- Evitar usar la cuenta LocalSystem, NetworkService o la cuenta del administrador.

Proyecto 2: Monitoreo de servicios críticos del servidor

Enunciado:

Implemente consultas administrativas que permitan verificar el estado de las instancias SQL Server relacionadas con GradosTítulos.

-- 1. Verificar el estado de los servicios SQL Server y Agent (desde T-SQL)

```
SELECT
    servicename,
    service_account,
    status_desc,
    startup_type_desc,
    process_id,
    last_startup_time
FROM sys.dm_server_services;
GO
```

-- 2. Verificar la conectividad y estado de la base de datos GradosTítulos
-- (Comprobar que la base de datos esté en línea)

```
SELECT name, state_desc, recovery_model_desc, compatibility_level
FROM sys.databases
WHERE name = 'GradosTítulos';
GO
```

-- 3. Verificar el tiempo de actividad del servidor

```
SELECT
    sqlserver_start_time,
    DATEDIFF(hour, sqlserver_start_time, GETDATE()) AS horas_activo
FROM sys.dm_os_sys_info;
GO
```

-- 4. Monitorear el uso de CPU y memoria (para detectar problemas)

```
SELECT
    (SELECT COUNT(*) FROM sys.dm_exec_requests WHERE status NOT IN
    ('background', 'sleeping')) AS solicitudes_activas,
    (SELECT COUNT(*) FROM sys.dm_os_waiting_tasks) AS tareas_espera,
    (SELECT total_physical_memory_kb / 1024 FROM sys.dm_os_sys_memory) AS
memoria_total_MB,
    (SELECT available_physical_memory_kb / 1024 FROM sys.dm_os_sys_memory) AS
memoria_disponible_MB;
GO
```

-- 5. Verificar el estado de los jobs del SQL Server Agent (si hay jobs para GradosTítulos)

```
SELECT
    j.name AS JobName,
    ja.run_status,
```

```

ja.last_run_date,
ja.last_run_time,
CASE ja.run_status
    WHEN 0 THEN 'Failed'
    WHEN 1 THEN 'Succeeded'
    WHEN 2 THEN 'Retry'
    WHEN 3 THEN 'Cancelled'
    ELSE 'Unknown'
END AS StatusDesc
FROM msdb.dbo.sysjobs j
LEFT JOIN msdb.dbo.sysjobactivity ja ON j.job_id = ja.job_id
WHERE j.enabled = 1;
GO

-- 6. Verificar la conectividad a la base de datos desde aplicaciones (usando
una tabla de prueba)
-- Crear una tabla de prueba (opcional) y hacer un simple SELECT.
-- También se puede usar el comando xp_cmdshell para probar ping, pero no
recomendado.

-- 7. Script de monitoreo que se puede ejecutar periódicamente
CREATE OR ALTER PROCEDURE Admin.SP_CheckServerStatus
AS
BEGIN
    -- Estado servicios
    SELECT 'Servicios SQL' AS Tipo, servicename, status_desc
    FROM sys.dm_server_services;

    -- Estado base de datos
    SELECT 'Base de datos' AS Tipo, name, state_desc
    FROM sys.databases WHERE name = 'GradosTitulos';

    -- Tiempo activo
    SELECT 'Uptime' AS Tipo,
        DATEDIFF(day, sqlserver_start_time, GETDATE()) AS DiasActivo,
        DATEDIFF(hour, sqlserver_start_time, GETDATE()) AS HorasActivo
    FROM sys.dm_os_sys_info;

    -- Recursos
    SELECT 'Memoria' AS Tipo,
        (available_physical_memory_kb / 1024) AS MemoriaDisponibleMB
    FROM sys.dm_os_sys_memory;
END;
GO

-- Ejemplo de ejecución
EXEC Admin.SP_CheckServerStatus;

```

Justificación técnica

El monitoreo proactivo de los servicios y del estado del servidor es esencial para garantizar la disponibilidad y el rendimiento de la base de datos en producción. Las consultas proporcionadas permiten verificar rápidamente que los servicios estén en ejecución, que la base de datos esté en línea, y que los recursos (CPU, memoria) sean suficientes. También se incluye el estado de los trabajos del Agente, que pueden ser críticos para backups y mantenimiento. Estos scripts pueden integrarse con herramientas de alerta (como correo electrónico) para notificar anomalías.

Buenas prácticas

- Establecer un monitoreo continuo con herramientas como SQL Server Management Studio, PowerShell o soluciones de terceros
- Programar la ejecución periódica de scripts de verificación y almacenar los resultados en una tabla de historial para análisis de tendencias.
- Configurar alertas por correo electrónico o SMS para fallos críticos (servicio detenido, base de datos en sospecha, etc.).
- Revisar los logs de errores de SQL Server periódicamente.
- Asegurarse de que los jobs de mantenimiento (backups, índices) se ejecuten correctamente.

Proyecto 3: Configuración de protocolos de red seguros

Enunciado:

Configure la instancia SQL Server para permitir únicamente conexiones seguras a la base de datos GradosTitulos.

```
-- 1. Verificar los protocolos habilitados (usando DMV o registro)
-- Consulta para ver el puerto y protocolo usado:
SELECT
    local_tcp_port,
    protocol_type,
    protocol_desc,
    encryption_option_desc
FROM sys.dm_exec_connections
WHERE session_id = @@SPID;
GO

-- 2. Instrucciones para configurar protocolos (usar SQL Server Configuration
Manager):
-- a) Habilitar solo TCP/IP y deshabilitar Named Pipes y Shared Memory (si no
se necesita).
-- b) Configurar el puerto TCP/IP estático (por ejemplo, 1433 o un puerto no
estándar).
-- c) Forzar el cifrado de conexiones (Encrypt = Yes) y usar certificados
SSL/TLS.
-- d) Opcional: Configurar el protocolo para usar solo TLS 1.2 (a nivel de
sistema operativo).

-- 3. Verificar que las conexiones usen cifrado:
-- Ejecutar la consulta anterior para ver encryption_option_desc.
-- Debe aparecer 'TRUE' si se ha forzado el cifrado.

-- 4. Configurar el login para exigir conexiones cifradas (opcional):
-- Para cada login, se puede forzar el cifrado usando ALTER LOGIN ... WITH
ENCRYPTION = REQUIRED? (No existe en SQL Server 2019+; se usa el nivel de
servidor).

-- 5. Crear un trigger de logon que rechace conexiones sin cifrado (si es
necesario)
-- Pero no es común; mejor forzar a nivel de servidor.

-- 6. Verificar la configuración de certificados (usar DMV)
SELECT * FROM sys.certificates; -- Para ver los certificados instalados.

-- 7. Recomendación adicional: Usar Always Encrypted para datos sensibles, pero
es opcional.
```

Justificación técnica

Las conexiones no cifradas exponen los datos transmitidos (incluyendo credenciales) a ataques de interceptación (man-in-the-middle). En entornos de producción, especialmente con datos sensibles como los de grados y títulos, es obligatorio usar cifrado SSL/TLS. Deshabilitar protocolos antiguos (Named Pipes, Shared Memory) reduce la superficie de ataque y fuerza el uso de TCP/IP que es más robusto y configurable. El uso de certificados autofirmados o de una CA empresarial garantiza la autenticación del servidor.

Buenas prácticas

Habilitar solo TCP/IP y, si es posible, usar un puerto no estándar para evitar escaneos automáticos.

- Forzar el cifrado de todas las conexiones a nivel de instancia.
- Usar certificados SSL firmados por una autoridad de confianza (no autofirmados en producción) para evitar advertencias de seguridad.
- Mantener actualizados los protocolos TLS (deshabilitar SSL 3.0 y TLS 1.0/1.1).
- Monitorear las conexiones para asegurar que todas usen cifrado.
- Aplicar políticas de firewall para restringir el acceso al puerto de SQL Server solo a direcciones IP autorizadas.

TEMA 3: Creación de roles fijos y personalizados

A continuación, se desarrollan los tres proyectos solicitados sobre la base de datos GradosTítulos. Se incluye el código SQL necesario y, al final, una tabla resumen con la justificación técnica y las buenas prácticas para cada proyecto.

Proyecto 1: Roles para el proceso de grados y títulos
Enunciado: Diseñe roles personalizados para gestionar el acceso a los esquemas: Catalogo, Academico, Tramite, Evaluacion, Documento.

```
USE GradosTítulos;  
GO
```

```
-- 1. Crear roles personalizados por esquema  
CREATE ROLE RolCatalogo;  
CREATE ROLE RolAcademico;  
CREATE ROLE RolTramite;  
CREATE ROLE RolEvaluacion;  
CREATE ROLE RolDocumento;  
  
-- 2. Asignar permisos según el esquema  
-- Esquema Catalogo (tablas de catálogo: solo lectura para la mayoría,  
-- escritura para administradores)  
GRANT SELECT ON SCHEMA::Catalogo TO RolCatalogo;  
-- Se puede ampliar con INSERT/UPDATE si se requiere  
  
-- Esquema Academico (gestión de personas, docentes, programas, etc.)  
GRANT SELECT, INSERT, UPDATE ON SCHEMA::Academico TO RolAcademico;  
-- Restringir DELETE solo a roles superiores  
  
-- Esquema Tramite (trámites, pagos, trabajo de investigación, etc.)
```

```

GRANT SELECT, INSERT, UPDATE, DELETE ON SCHEMA::Tramite TO RolTramite;

-- Esquema Evaluacion (evaluaciones, sustentaciones, etc.)
GRANT SELECT, INSERT, UPDATE ON SCHEMA::Evaluacion TO RolEvaluacion;

-- Esquema Documento (resoluciones, diplomas)
GRANT SELECT, INSERT, UPDATE ON SCHEMA::Documento TO RolDocumento;

-- 3. Crear roles compuestos si se necesita (ejemplo: coordinador tiene acceso
a todos)
CREATE ROLE CoordinadorGeneral;
GRANT SELECT, INSERT, UPDATE, DELETE ON SCHEMA::Catalogo TO CoordinadorGeneral;
GRANT SELECT, INSERT, UPDATE, DELETE ON SCHEMA::Academico TO CoordinadorGeneral;
GRANT SELECT, INSERT, UPDATE, DELETE ON SCHEMA::Tramite TO CoordinadorGeneral;
GRANT SELECT, INSERT, UPDATE, DELETE ON SCHEMA::Evaluacion TO
CoordinadorGeneral;
GRANT SELECT, INSERT, UPDATE, DELETE ON SCHEMA::Documento TO CoordinadorGeneral;

-- 4. Asignar usuarios de ejemplo (los creados en proyectos anteriores)
-- ALTER ROLE RolTramite ADD MEMBER [usuario_ejemplo];
-- ALTER ROLE CoordinadorGeneral ADD MEMBER [coordinador_gt];

```

Justificacion tecnica

Segmentar permisos por esquema facilita la administración y reduce el riesgo de sobrepermisos. Permite asignar accesos granulares según la responsabilidad (ej. solo lectura para catálogos, escritura para trámites).

Buenas practicas

- Usar el principio de mínimo privilegio
- Crear roles específicos y no asignar permisos directamente a usuarios.
- Revisar periódicamente las membresías de roles.

Proyecto 2: Implementación de roles para evaluadores de tesis
Enunciado: Los evaluadores deben consultar tesis y registrar observaciones sin modificar información académica.

```

USE GradosTitulos;
GO

-- 1. Crear rol para evaluadores
CREATE ROLE EvaluadorTesis;

-- 2. Permisos de consulta sobre tablas de tesis y sustentación
GRANT SELECT ON Tramite.TrabajoInvestigacion TO EvaluadorTesis;
GRANT SELECT ON Tramite.DesignacionJurado TO EvaluadorTesis;
GRANT SELECT ON Tramite.DesignacionAsesor TO EvaluadorTesis;
GRANT SELECT ON Evaluacion.Sustentacion TO EvaluadorTesis;
GRANT SELECT ON Evaluacion.EvaluacionTrabajo TO EvaluadorTesis;
GRANT SELECT ON Evaluacion.ControlSimilitud TO EvaluadorTesis;
GRANT SELECT ON Evaluacion.DictamenEtica TO EvaluadorTesis;

-- 3. Permisos para registrar observaciones (INSERT/UPDATE en tabla
ObservacionTramite)
GRANT INSERT, UPDATE ON Tramite.ObservacionTramite TO EvaluadorTesis;
-- También pueden actualizar el estado de la observación (por ejemplo, marcar
como subsanado)

```

```

-- 4. Restringir acceso a tablas sensibles (personas, matrículas, pagos)
DENY SELECT ON Academico.Persona TO EvaluadorTesis;
DENY SELECT ON Academico.Matricula TO EvaluadorTesis;
DENY SELECT ON Tramite.Pago TO EvaluadorTesis;
DENY SELECT ON Documento.Diploma TO EvaluadorTesis;

-- 5. Opcional: crear una vista específica para evaluadores que filtre datos
CREATE VIEW Evaluacion.V_TesisParaEvaluador
AS
SELECT t.IdTrabajo, t.Titulo, t.FechaInicio, t.FechaFin, t.ModalidadAutoria,
       l.NombreLinea, p.Nombres AS NombreEstudiante, p.ApellidoPaterno
FROM Tramite.TrabajoInvestigacion t
JOIN Tramite.Tramite tr ON t.IdTramite = tr.IdTramite
JOIN Academico.Matricula m ON tr.IdMatricula = m.IdMatricula
JOIN Academico.Persona p ON m.IdEstudiante = p.IdPersona
JOIN Catalogo.LineaInvestigacion l ON t.IdLinea = l.IdLinea;
GRANT SELECT ON Evaluacion.V_TesisParaEvaluador TO EvaluadorTesis;

```

Justificacion Tecnica

Los evaluadores necesitan leer datos de tesis y registrar observaciones, pero no deben modificar información académica ni personal. Separar estos permisos evita alteraciones accidentales o maliciosas.

Buenas Practicas

- Denegar explícitamente el acceso a datos sensibles.
- Crear vistas para mostrar solo los datos necesarios.
- Auditar los cambios en observaciones.

Proyecto 3: Administración de roles jerárquicos Enunciado: Configure roles que representen los niveles: Operador, Supervisor, Administrador para el sistema de grados y títulos.

```

USE GradosTitulos;
GO

-- 1. Crear roles jerárquicos
CREATE ROLE Operador;
CREATE ROLE Supervisor;
CREATE ROLE Administrador;

-- 2. Asignar permisos según nivel

-- Operador: puede gestionar trámites (crear, actualizar) y consultar
información básica
GRANT SELECT, INSERT, UPDATE ON Tramite.Tramite TO Operador;
GRANT SELECT, INSERT, UPDATE ON Tramite.Pago TO Operador;
GRANT SELECT, INSERT, UPDATE ON Tramite.Fotografia TO Operador;
GRANT SELECT ON Academico.Persona TO Operador;
GRANT SELECT ON Academico.Matricula TO Operador;
GRANT SELECT ON Catalogo.TipoTramite TO Operador;
GRANT SELECT ON Catalogo.EstadoTramite TO Operador;
-- Sin DELETE ni permisos sobre evaluación o documentos

-- Supervisor: además de lo de operador, puede aprobar, revisar y gestionar
evaluaciones
GRANT DELETE ON Tramite.Tramite TO Supervisor; -- para archivar o anular

```



```

GRANT SELECT, INSERT, UPDATE, DELETE ON Evaluacion.EvaluacionTrabajo TO
Supervisor;
GRANT SELECT, INSERT, UPDATE, DELETE ON Evaluacion.Sustentacion TO Supervisor;
GRANT SELECT, INSERT, UPDATE, DELETE ON Evaluacion.ControlSimilitud TO
Supervisor;
GRANT SELECT, INSERT, UPDATE, DELETE ON Evaluacion.DictamenEtica TO Supervisor;
GRANT SELECT, INSERT, UPDATE ON Documento.Resolucion TO Supervisor;
GRANT SELECT, INSERT, UPDATE ON Documento.Diploma TO Supervisor;
-- Puede modificar catálogos básicos
GRANT INSERT, UPDATE ON Catalogo.LineaInvestigacion TO Supervisor;

-- Administrador: todos los permisos sobre todos los esquemas
GRANT CONTROL ON SCHEMA::Catalogo TO Administrador;
GRANT CONTROL ON SCHEMA::Academico TO Administrador;
GRANT CONTROL ON SCHEMA::Tramite TO Administrador;
GRANT CONTROL ON SCHEMA::Evaluacion TO Administrador;
GRANT CONTROL ON SCHEMA::Documento TO Administrador;
-- También puede administrar roles y usuarios
GRANT ALTER ANY ROLE TO Administrador;
GRANT VIEW DEFINITION TO Administrador;

-- 3. Asignar usuarios de ejemplo (los creados en proyectos anteriores)
-- ALTER ROLE Operador ADD MEMBER [secretario_gt];
-- ALTER ROLE Supervisor ADD MEMBER [coordinador_gt];
-- ALTER ROLE Administrador ADD MEMBER [admin_grados];

```

Justificacion Tecnica

Representa los niveles de responsabilidad (Operador, Supervisor, Administrador). Permite escalar permisos de forma progresiva y facilita el control de acceso basado en el puesto.

Buenas Practicas

- Asignar roles según el puesto y no según la persona.
- Mantener una matriz de permisos documentada.
- Restringir el rol Administrador a un número muy reducido de cuentas y habilitar auditoría para sus acciones.

TEMA 4: Control de acceso mediante GRANT, DENY y REVOKE

continuación se desarrollan los tres proyectos sobre la base de datos GradosTitulos, utilizando las instrucciones GRANT, DENY y REVOKE para gestionar los permisos de forma granular.

Proyecto 1: Operadores consultan datos de estudiantes pero no los modifican

Enunciado: Los operadores pueden consultar datos de estudiantes, pero no modificarlos.

```
USE GradosTitulos;  
GO
```

```
-- 1. Crear el rol de operador (si no existe)
```

```
CREATE ROLE OperadorEstudiantes;
```

```
-- 2. Conceder permiso de SELECT sobre las tablas de estudiantes
```

```
GRANT SELECT ON Academico.Persona TO OperadorEstudiantes;
```

```
GRANT SELECT ON Academico.Matricula TO OperadorEstudiantes;
```

```
-- 3. Denegar explícitamente operaciones de modificación (INSERT, UPDATE,  
DELETE)
```

```
-- sobre las mismas tablas
```

```
DENY INSERT, UPDATE, DELETE ON Academico.Persona TO OperadorEstudiantes;
```

```
DENY INSERT, UPDATE, DELETE ON Academico.Matricula TO OperadorEstudiantes;
```

```
-- 4. Asignar usuarios al rol (ejemplo)
```

```
-- ALTER ROLE OperadorEstudiantes ADD MEMBER [secretario_gt];
```

```
-- ALTER ROLE OperadorEstudiantes ADD MEMBER [usuario_operador];
```

Justificación técnica

Se aplica el principio de mínimo privilegio: los operadores necesitan leer información de estudiantes para atender consultas, pero no deben alterar datos (p. ej., cambiar nombres, notas o deudas). Usar DENY refuerza la prohibición incluso si otros roles heredaran permisos de escritura.

Buenas prácticas

- Usar DENY solo cuando sea estrictamente necesario; normalmente es mejor simplemente no otorgar permisos.
- Documentar las denegaciones para evitar confusiones.
- Revisar periódicamente los per

Proyecto 2: Restricción de acceso a resoluciones y diplomas

Enunciado: Solo la Oficina de Grados y Títulos puede modificar documentos oficiales (resoluciones y diplomas).

```
USE GradosTitulos;
GO

-- 1. Crear roles para la Oficina de Grados y Títulos y para el resto de
usuarios
CREATE ROLE OficinaGT;
CREATE ROLE UsuariosGenerales;

-- 2. Otorgar permisos completos (SELECT, INSERT, UPDATE, DELETE) al rol
OficinaGT
GRANT SELECT, INSERT, UPDATE, DELETE ON Documento.Resolucion TO OficinaGT;
GRANT SELECT, INSERT, UPDATE, DELETE ON Documento.Diploma TO OficinaGT;

-- 3. A los demás usuarios (UsuariosGenerales) se les concede solo SELECT
GRANT SELECT ON Documento.Resolucion TO UsuariosGenerales;
GRANT SELECT ON Documento.Diploma TO UsuariosGenerales;

-- 4. Denegar explícitamente las operaciones de modificación a
UsuariosGenerales
DENY INSERT, UPDATE, DELETE ON Documento.Resolucion TO UsuariosGenerales;
DENY INSERT, UPDATE, DELETE ON Documento.Diploma TO UsuariosGenerales;

-- 5. Asignar miembros a los roles
-- ALTER ROLE OficinaGT ADD MEMBER [coordinador_gt];
-- ALTER ROLE OficinaGT ADD MEMBER [admin_grados];
-- ALTER ROLE UsuariosGenerales ADD MEMBER [secretario_gt]; -- si no es de la
oficina
-- Nota: Si un usuario pertenece a OficinaGT, no se le aplica el DENY de
UsuariosGenerales,
-- porque los DENY se evalúan por rol y el usuario puede tener permisos
positivos de otro rol.
-- Para asegurar que solo OficinaGT pueda modificar, conviene que los demás
roles no tengan
-- permisos de modificación (como se hace aquí con GRANT solo SELECT y DENY).
```

Justificación técnica:

Se aplica el principio de mínimo privilegio: los operadores necesitan leer información de estudiantes para atender consultas, pero no deben alterar datos (p. ej., cambiar nombres, notas o deudas). Usar DENY refuerza la prohibición incluso si otros roles heredaran permisos de escritura.

Buenas prácticas:

- Asignar permisos de modificación solo a un rol específico y denegarlos a los demás.
- Mantener un registro de auditoría de cambios en documentos oficiales.
- Utilizar GRANT para dar acceso y DENY para bloquear explícitamente cuando sea necesario.

Proyecto 3: Revocación de permisos temporales
Enunciado: Un auditor externo requiere acceso temporal durante una semana.

```
USE GradosTitulos;
GO

-- 1. Crear un usuario/login para el auditor externo (si no existe)
-- (Asumimos que ya se creó con autenticación SQL)
-- Ejemplo: CREATE LOGIN auditor_ext WITH PASSWORD = '...';
-- CREATE USER auditor_ext FOR LOGIN auditor_ext;

-- 2. Otorgar permisos necesarios para la auditoría
GRANT SELECT ON Tramite.Tramite TO auditor_ext;
GRANT SELECT ON Tramite.Pago TO auditor_ext;
GRANT SELECT ON Documento.Resolucion TO auditor_ext;
GRANT SELECT ON Documento.Diploma TO auditor_ext;
GRANT SELECT ON Academico.Persona TO auditor_ext;
GRANT SELECT ON Academico.Matricula TO auditor_ext;

-- 3. Programar la revocación de permisos después de 7 días
-- Opción A: Crear un trabajo de SQL Agent que ejecute la revocación
-- Opción B: Ejecutar manualmente el siguiente comando al final del período

-- Comando de revocación (se ejecutará al séptimo día):
REVOKE SELECT ON Tramite.Tramite FROM auditor_ext;
REVOKE SELECT ON Tramite.Pago FROM auditor_ext;
REVOKE SELECT ON Documento.Resolucion FROM auditor_ext;
REVOKE SELECT ON Documento.Diploma FROM auditor_ext;
REVOKE SELECT ON Academico.Persona FROM auditor_ext;
REVOKE SELECT ON Academico.Matricula FROM auditor_ext;

-- 4. Alternativa: Usar un procedimiento almacenado que verifique la fecha y
revoque automáticamente
-- (Ejemplo de un job diario que ejecute la revocación si ha pasado la fecha
límite)
```

Justificación técnica

La auditoría externa requiere permisos adicionales por un período limitado. La revocación oportuna elimina los permisos concedidos, evitando que el auditor mantenga acceso después de finalizar su trabajo.

Buenas prácticas

- Planificar la revocación con antelación (usar trabajos programados).
- No conceder permisos permanentes a cuentas temporales.
- Registrar la concesión y revocación en un log de seguridad.
- Usar REVOKE en lugar de DENY para eliminar permisos que se concedieron específicamente.

TEMA 5: Cifrado y protección de datos (TDE y Always Encrypted)

A continuación se presentan los scripts para implementar Transparent Data Encryption (TDE) y Always Encrypted en la base de datos GradosTitulos, junto con la justificación y buenas prácticas resumidas

Proyecto 1: Implementación de TDE para GradosTitulos

Enunciado:

Proteja los archivos físicos de la base de datos mediante Transparent Data Encryption.

```
-- 1. En la base de datos master, crear la clave maestra y el certificado
USE master;
GO

CREATE MASTER KEY ENCRYPTION BY PASSWORD = 'Upla#2026!TDE_MasterKey';
GO

CREATE CERTIFICATE CertGradosTitulos
    WITH SUBJECT = 'Certificado para TDE en GradosTitulos';
GO

-- 2. Cambiar a la base de datos GradosTitulos y crear la clave de cifrado de
base de datos
USE GradosTitulos;
GO

CREATE DATABASE ENCRYPTION KEY
    WITH ALGORITHM = AES_256
    ENCRYPTION BY SERVER CERTIFICATE CertGradosTitulos;
GO

-- 3. Activar el cifrado en la base de datos
ALTER DATABASE GradosTitulos
SET ENCRYPTION ON;
GO

-- 4. Verificar el estado del cifrado
SELECT name, is_encrypted
FROM sys.databases
WHERE name = 'GradosTitulos';
```

Justificación técnica

Protege los datos en reposo (archivos .mdf, .ldf, backups) cifrando todo el contenido de la base de datos a nivel de página. No afecta a las aplicaciones ni a los usuarios, ya que el descifrado ocurre de forma transparente al leer en memoria.

Buenas prácticas

- Realizar copias de seguridad de la clave maestra y el certificado y almacenarlas en un lugar seguro.
- Monitorear el rendimiento, ya que TDE añade una pequeña sobrecarga de CPU.
- Activar TDE antes de poner la base en producción.

Proyecto 2: Protección de datos personales del tesista

Enunciado:

Implemente Always Encrypted para proteger DNI, Correo electrónico y Teléfono de los estudiantes (tabla Persona)

Paso 1: Crear un certificado en el almacén de Windows (ejecutar en PowerShell como administrador):

```
# Crear un certificado auto-firmado en el almacén CurrentUser/My
$cert = New-SelfSignedCertificate -Subject "CN=AlwaysEncrypted_Grados" -
CertStoreLocation "Cert:\CurrentUser\My" -KeyExportPolicy Exportable -Type
DocumentEncryptionCert -KeyUsage KeyEncipherment -KeySpec KeyExchange
# Obtener la huella
$cert.Thumbprint
```

Paso 2: En SQL Server, crear la clave maestra de columna (CMK) y la clave de cifrado de columna (CEK):

```
USE GradosTitulos;
GO

-- Crear la clave maestra de columna referenciando el certificado (reemplazar
'XXXXXXXX' por la huella)
CREATE COLUMN MASTER KEY [CMK_Grados]
WITH (
    KEY_STORE_PROVIDER_NAME = N'MSSQL_CERTIFICATE_STORE',
    KEY_PATH = N'CurrentUser/My/XXXXXXXX' -- Huella del certificado
);
GO

-- Crear la clave de cifrado de columna (CEK) con algoritmo AES_256
CREATE COLUMN ENCRYPTION KEY [CEK_Persona]
WITH VALUES (
    COLUMN_MASTER_KEY = [CMK_Grados],
    ALGORITHM = 'RSA_OAEP',
    ENCRYPTED_VALUE = 0x... -- Valor generado automáticamente por SSMS al
cifrar columnas
);
-- Nota: La CEK se genera automáticamente al usar el asistente de Always
Encrypted en SSMS.
-- Se muestra el script generado por el asistente para referencia.
```

Paso 3: Modificar las columnas de la tabla Persona para aplicar el cifrado (usando el asistente de SSMS o el siguiente script generado):

```
ALTER TABLE Academico.Persona
ALTER COLUMN DNI CHAR(8) COLLATE Latin1_General_BIN2
ENCRYPTED WITH (COLUMN_ENCRYPTION_KEY = [CEK_Persona], ENCRYPTION_TYPE =
Deterministic, ALGORITHM = 'AEAD_AES_256_CBC_HMAC_SHA_256');
GO
```

```
ALTER TABLE Academico.Persona
ALTER COLUMN Correo VARCHAR(120) COLLATE Latin1_General_BIN2
ENCRYPTED WITH (COLUMN_ENCRYPTION_KEY = [CEK_Persona], ENCRYPTION_TYPE =
Deterministic, ALGORITHM = 'AEAD_AES_256_CBC_HMAC_SHA_256');
GO
```

```
ALTER TABLE Academico.Persona
ALTER COLUMN Telefono VARCHAR(15) COLLATE Latin1_General_BIN2
```

```

ENCRYPTED WITH (COLUMN_ENCRYPTION_KEY = [CEK_Persona], ENCRYPTION_TYPE =
Randomized, ALGORITHM = 'AEAD_AES_256_CBC_HMAC_SHA_256');
GO

```

Justificación técnica

Protege los datos en reposo (archivos .mdf, .ldf, backups) cifrando todo el contenido de la base de datos a nivel de página. No afecta a las aplicaciones ni a los usuarios, ya que el descifrado ocurre de forma transparente al leer en memoria.

Buenas prácticas

- Realizar copias de seguridad de la clave maestra y el certificado y almacenarlas en un lugar seguro
- Monitorear el rendimiento, ya que TDE añade una pequeña sobrecarga de CPU.
- Activar TDE antes de poner la base en producción.

Proyecto 3: Cifrado de información de jurados y asesores

Enunciado:

Proteja la información personal de asesores y jurados de tesis.

Los asesores y jurados son docentes registrados en la tabla Persona (con TipoPersona = 'DOCENTE'). Por lo tanto, las columnas DNI, Correo y Teléfono ya quedan protegidas con el proyecto 2. Adicionalmente, se puede cifrar el campo CodigOrcid de la tabla Docente, que es un identificador único personal.

```

-- Cifrar la columna CodigOrcid en la tabla Docente (usando la misma CEK o una
nueva)
ALTER TABLE Academico.Docente
ALTER COLUMN CodigOrcid VARCHAR(25) COLLATE Latin1_General_BIN2
ENCRYPTED WITH (COLUMN_ENCRYPTION_KEY = [CEK_Persona], ENCRYPTION_TYPE =
Deterministic, ALGORITHM = 'AEAD_AES_256_CBC_HMAC_SHA_256');
GO

```

Justificación técnica

Extiende la protección a identificadores personales de los docentes (ORCID), asegurando que incluso si la base de datos es comprometida, esta información no pueda ser leída sin la clave correspondiente.

Buenas prácticas

- Aplicar el mismo criterio de cifrado que para estudiantes.
- Documentar qué columnas están cifradas y con qué tipo de cifrado.
- Asegurar que las aplicaciones cliente soporten Always Encrypted (por ejemplo, con .NET Framework 4.6+ o drivers actualizados).

TEMA 6: Auditoría y monitoreo de eventos con SQL Server Audit

A continuación se presentan los scripts para implementar auditoría en la base de datos GradosTitulos utilizando SQL Server Audit. Se incluyen tres proyectos que cubren desde el registro de accesos hasta una auditoría integral.

Proyecto 1: Auditoría de accesos a la base de datos

Enunciado:

Registrar todos los accesos realizados a la base de datos GradosTitulos.

```
-- 1. Crear el objeto de auditoría a nivel de servidor
USE master;
GO

CREATE SERVER AUDIT Audit_Accesos_Grados
TO FILE (
    FILEPATH = 'C:\AuditLogs\GradosTitulos\',
    MAXSIZE = 200 MB,
    MAX_ROLLOVER_FILES = 10,
    RESERVE_DISK_SPACE = OFF
)
WITH (
    QUEUE_DELAY = 1000,
    ON_FAILURE = CONTINUE
);
GO

ALTER SERVER AUDIT Audit_Accesos_Grados WITH (STATE = ON);
GO

-- 2. Crear especificación de auditoría del servidor para eventos de inicio de sesión
CREATE SERVER AUDIT SPECIFICATION Spec_Accesos_Servidor
FOR SERVER AUDIT Audit_Accesos_Grados
ADD (SUCCESSFUL_LOGIN_GROUP),
ADD (FAILED_LOGIN_GROUP),
ADD (LOGIN_CHANGE_PASSWORD_GROUP); -- opcional: cambios de contraseña
GO

ALTER SERVER AUDIT SPECIFICATION Spec_Accesos_Servidor WITH (STATE = ON);
GO

-- 3. Crear especificación de auditoría a nivel de base de datos para cambios de base de datos
USE GradosTitulos;
GO

CREATE DATABASE AUDIT SPECIFICATION Spec_Accesos_BaseDatos
FOR SERVER AUDIT Audit_Accesos_Grados
ADD (DATABASE_OBJECT_CHANGE_GROUP), -- creación/alteración/eliminación de objetos
ADD (DATABASE_CHANGE_GROUP), -- cambios en la base de datos (ej. ALTER DATABASE)
ADD (DATABASE_PRINCIPAL_CHANGE_GROUP); -- creación/alteración de usuarios/roles
GO

ALTER DATABASE AUDIT SPECIFICATION Spec_Accesos_BaseDatos WITH (STATE = ON);
GO
```


Justificación técnica

Registrar quién, cuándo y desde dónde se conecta a la base de datos es fundamental para la seguridad y para cumplir con normativas (GDPR, Ley de Datos Personales). Permite detectar accesos no autorizados o fuera de horario.

Buenas prácticas

- Almacenar los archivos de auditoría en una ubicación segura y con permisos restringidos.
- Monitorear el tamaño de los archivos y rotarlos.
- Configurar ON_FAILURE = CONTINUE para no interrumpir el servicio si falla la escritura.

Proyecto 2: Auditoría de modificaciones en trámites académicos

Enunciado:

Registrar todas las operaciones INSERT, UPDATE y DELETE realizadas sobre las tablas del esquema Tramite.

```
-- 1. Crear auditoría (si no se creó en el proyecto anterior, se puede
reutilizar)
-- Si ya existe Audit_Accesos_Grados, se puede usar para este proyecto también.
-- En este caso, creamos una nueva auditoría específica para DML.
USE master;
GO

CREATE SERVER AUDIT Audit_DML_Tramite
TO FILE (
    FILEPATH = 'C:\AuditLogs\GradosTitulos_DML\' ,
    MAXSIZE = 500 MB,
    MAX_ROLLOVER_FILES = 20
)
WITH (QUEUE_DELAY = 1000, ON_FAILURE = CONTINUE);
GO

ALTER SERVER AUDIT Audit_DML_Tramite WITH (STATE = ON);
GO

-- 2. Crear especificación de auditoría de base de datos para el esquema
Tramite
USE GradosTitulos;
GO

CREATE DATABASE AUDIT SPECIFICATION Spec_DML_Tramite
FOR SERVER AUDIT Audit_DML_Tramite
ADD (INSERT ON SCHEMA::Tramite BY PUBLIC),
ADD (UPDATE ON SCHEMA::Tramite BY PUBLIC),
ADD (DELETE ON SCHEMA::Tramite BY PUBLIC);
GO

ALTER DATABASE AUDIT SPECIFICATION Spec_DML_Tramite WITH (STATE = ON);
GO

-- 3. (Opcional) Si se quiere registrar también SELECT sobre tablas sensibles,
-- se puede añadir: ADD (SELECT ON Tramite.Tramite BY PUBLIC) pero podría
generar mucho volumen.
```

Justificación técnica

Las tablas del esquema Tramite contienen información sensible (pagos, estados, etc.). Registrar modificaciones permite auditar cambios indebidos y tener un historial para investigaciones.

Buenas prácticas

- Evitar auditar SELECT a menos que sea estrictamente necesario (genera mucho volumen).
- Usar BY PUBLIC para auditar a todos los usuarios, o filtrar por roles específicos.
- Incluir índices sobre las columnas de fecha en las tablas de auditoría si se usan funciones personalizadas.

Proyecto 3: Sistema integral de auditoría institucional

Enunciado:

Diseñar una solución de auditoría que registre: Inicio de sesión. Cambios de permisos. Creación de usuarios. Eliminación de objetos. Modificaciones de datos críticos.

```
-- 1. Crear una auditoría integrada (puede ser la misma que la del proyecto 1, pero ampliada)
USE master;
GO
```

```
CREATE SERVER AUDIT Audit_Integral_Grados
TO FILE (
    FILEPATH = 'C:\AuditLogs\GradosTitulos_Integral\' ,
    MAXSIZE = 1 GB,
    MAX_ROLLOVER_FILES = 50,
    RESERVE_DISK_SPACE = OFF
)
WITH (QUEUE_DELAY = 1000, ON_FAILURE = SHUTDOWN); -- En producción se podría usar CONTINUE
GO
```

```
ALTER SERVER AUDIT Audit_Integral_Grados WITH (STATE = ON);
GO
```

```
-- 2. Especificación de auditoría del servidor: inicios de sesión, cambios de permisos, creación de usuarios
CREATE SERVER AUDIT SPECIFICATION Spec_Integral_Servidor
FOR SERVER AUDIT Audit_Integral_Grados
ADD (SUCCESSFUL_LOGIN_GROUP),
ADD (FAILED_LOGIN_GROUP),
ADD (LOGOUT_GROUP), -- cierre de sesión
ADD (SERVER_PRINCIPAL_CHANGE_GROUP), -- creación/alteración/eliminación de logins
ADD (SERVER_ROLE_MEMBER_CHANGE_GROUP), -- cambios en membresías de roles de servidor
ADD (SERVER_PERMISSION_CHANGE_GROUP); -- cambios de permisos a nivel de servidor (GRANT/REVOKE/DENY)
GO
```

```
ALTER SERVER AUDIT SPECIFICATION Spec_Integral_Servidor WITH (STATE = ON);
GO
```

```
-- 3. Especificación de auditoría de base de datos: creación de objetos (usuarios), eliminación de objetos,
```

```

-- modificaciones de datos críticos (ej. tablas de trámites, resoluciones,
diplomas)
USE GradosTitulos;
GO

CREATE DATABASE AUDIT SPECIFICATION Spec_Integral_BaseDatos
FOR SERVER AUDIT Audit_Integral_Grados
ADD (DATABASE_PRINCIPAL_CHANGE_GROUP),      -- creación/alteración/eliminación
de usuarios/roles en BD
ADD (DATABASE_OBJECT_CHANGE_GROUP),         -- creación/alteración/eliminación
de objetos (tablas, vistas, etc.)
ADD (SCHEMA_OBJECT_CHANGE_GROUP),          -- cambios en esquemas
-- Modificaciones de datos críticos (INSERT, UPDATE, DELETE en tablas
específicas)
ADD (INSERT ON Trámite.Trámite BY PUBLIC),
ADD (UPDATE ON Trámite.Trámite BY PUBLIC),
ADD (DELETE ON Trámite.Trámite BY PUBLIC),
ADD (INSERT ON Documento.Resolucion BY PUBLIC),
ADD (UPDATE ON Documento.Resolucion BY PUBLIC),
ADD (DELETE ON Documento.Resolucion BY PUBLIC),
ADD (INSERT ON Documento.Diploma BY PUBLIC),
ADD (UPDATE ON Documento.Diploma BY PUBLIC),
ADD (DELETE ON Documento.Diploma BY PUBLIC),
-- También podemos registrar cambios de permisos a nivel de base de datos
ADD (DATABASE_PERMISSION_CHANGE_GROUP);     -- GRANT/REVOKE/DENY en la BD
GO

ALTER DATABASE AUDIT SPECIFICATION Spec_Integral_BaseDatos WITH (STATE = ON);
GO

-- 4. Consulta para ver los registros de auditoría
SELECT
    event_time,
    action_id,
    succeeded,
    server_principal_name,
    database_principal_name,
    [object_name],
    [statement]
FROM sys.fn_get_audit_file('C:\AuditLogs\GradosTitulos_Integral\*.sqlaudit',
DEFAULT, DEFAULT)
ORDER BY event_time DESC;
GO

```

Justificación

Cubre todos los aspectos críticos: accesos, cambios de permisos, gestión de usuarios y objetos, y modificaciones de datos sensibles. Es la solución recomendada para cumplir con estándares de seguridad (ISO 27001, PCI-DSS) y facilitar auditorías externas.

técnica Buenas prácticas

- Centralizar todas las auditorías en un mismo destino para facilitar la consulta.
- Establecer políticas de retención (ej. conservar 90 días).
- Revisar periódicamente los logs con herramientas de análisis (Power BI, SIEM).
- Proteger los archivos de auditoría contra eliminación o modificación.
- Realizar pruebas de rendimiento antes de poner en producción, ya que la auditoría añade overhead.